

Assignment 1 Team 23

CINN_Assign1_[55-4895_55-5012_55-5066]

Team Members

Member 1 Name: Ahmed Samy Abed

Member 1 Tutorial: 2

Member 1 ID: 55-4895

Member 1 email: ahmed.abedibrahim@student.guc.edu.eg

Member 2 Name: Abdullah Hesham Eldaoushy

Member 2 Tutorial: 2

Member 2 ID: 55-5012

Member 2 email: abdullah.eldaoshy@student.guc.edu.eg

Member 3 Name: Youssef Raafat Malatawy

Member 3 Tutorial: 2

Member 3 ID: 55-5066

Member 3 email: yousef.elmalatawy@student.guc.edu.eg

Class definition

A Flower class was created to represent each individual in the population. The flower has a DNA sequence defined by 11 genes.

- Gene 1: `sizeOfCenter` the function that defines the center size by generating a random floating-point number between 1.0 and 3.0 and rounding to the nearest 2 decimal places
- Gene (2,3,4): `colorOfCenter` the 3 RGB color channels for the center calls the `randomColor` function to generate an (R, G, B) tuple for the center
- Gene (5,6,7): `colorOfPetals` the 3 RGB color channels for the petals calls the `randomColor` function to generate an (R, G, B) tuple for the petals
- Gene(8,9,10): `colorOfStem` the 3 RGB color channels for the stem calls the `randomColor` function a third time to generate an (R, G, B) tuple for the stem
- Gene 11: `numberOfPetals` the function that defines the number of petals by using a randomly generated number from 1 to 7

Those genes are combined in `Flower(sizeOfCenter_2, colorOfCenter_2, colorOfpetals_2, colorOfstem_2, numberOfPetals_2)` to create the flower.

Population Initialization

The `initPop` function is responsible for creating the initial, random population of flowers. It adheres to the required population size of 8 flowers. This initial population (Generation 1) is then printed to the console and displayed to the user.

```
def initPop (size , dictionary):
    population = []
    for i in range(size):
        center = round(random.uniform(1,3),2) # 2 decimal places
        color_center = randomColor()
        color_petals = randomColor()
        stem_color = randomColor()
        petals= random.randint(0,7)
        flower = Flower(center,color_center,color_petals,stem_color,petals)
        population.append(flower)
    dictionary[f"Generation{generation_number}"] = population
    print(f"--- Generation {generation_number} ---")
    for flower in population:
        flower.printDetails()
    print("="*50)
    return population
```

Fitness Evaluation

Fitness for each flower is determined by user interaction by hovering the mouse on the flower. This is implemented in the `draw_generation` function.

- The function uses `%matplotlib widget` to create an interactive plot displaying all 8 flowers in the current generation.
- A "motion_notify_event" is connected to the figure's canvas to track the mouse's position.
- A timer (`fig.canvas.new_timer`) is set to "tick" every 150ms.
- During each tick, the function checks which axes (which flower) the mouse is currently hovering over.
- If the mouse is over a flower, that flower's `fitness` attribute is incremented by 1.
- This mechanism directly links the hover duration to the fitness score. The updated fitness is displayed in real-time below each flower.

Genetic Algorithm Logic

The genetic algorithm logic uses selection, crossover, and mutation that are implemented as static methods within the `Flower` class and orchestrated by the `evolve` function

Selection

The selection process determines which flowers will become parents for the next generation. Our implementation uses an elitist strategy:

- **Sort:** The `Flower.sort` method sorts the entire population of 8 flowers in descending order based on their `fitness` attribute.
- **Select:** The `Flower.getTop4` method selects the four fittest flowers from the sorted list.
- **Duplicate:** The `Flower.duplicateAndShuffleTop4` method takes these top 4 individuals and duplicates them, creating a new "parent pool" to a total of 8 flowers. This pool is then shuffled.

- **Pair:** The `Flower.generateParentsPaired` method pairs up the 8 flowers in the shuffled pool, resulting in 4 parent pairs.

```
@staticmethod
def sort (generation):
    return sorted(generation, key=lambda flower: flower.getFitness(), reverse=True)

@staticmethod
def getTop4 (generation):
    sorted_generation = Flower.sort(generation)
    return sorted_generation[:4]

@staticmethod
def duplicateAndShuffleTop4 (top4):
    duped = []
    for flower in top4:
        duped.append(flower)
        duped.append(flower)
    random.shuffle(duped)
    return duped

@staticmethod
def generateParentsPaired (generation):
    parents = []
    top4 = Flower.getTop4(generation)
    ls = Flower.duplicateAndShuffleTop4(top4)
    for i in range(0, len(ls), 2):
        parents.append((ls[i], ls[i+1]))
    return parents
```

This strategy ensures that the fittest individuals are "more likely to be selected"; in fact, the top 4 are guaranteed to reproduce twice, while the bottom 4 are guaranteed not to be selected at all.

Crossover

Crossover combines the genetic information of two parents to create two new children.

- This is handled by the `Flower.crossover` method, which is called for each parent pair.

- A random number from 0 to 100 is generated. If this number is less than 65, crossover occurs. This implements the required 65% crossover probability. If the number is 65 or higher, the parents are returned unchanged.
- If crossover proceeds, a single `cutPoint` (an integer from 1 to 10) is randomly selected.
- Helper methods (`crossover_helper_middleOfColors` and `crossover_helper_endOfColors`) perform the single-point crossover. These helpers correctly handle the gene boundaries, even when the cut point falls *within* one of the 3-gene color tuples.
- The method includes detailed print statements showing the two parents and the two resulting children, as required.

```
@staticmethod
def crossover(parent1, parent2):
    cutpoints_inColors = [2,3,5,6,8,9]

    willCrossover = random.randint(0, 100)
    if willCrossover >= 65 :
        return parent1, parent2
    else :
        cutPoint = random.randint(1, 10)
        print(f"*** Crossover at gene {cutPoint} ***")
        print("---- Parent 1 ----")
        parent1.printDetails()
        print("---- Parent 2 ----")
        parent2.printDetails()

        if cutPoint in cutpoints_inColors :
            child1, child2 = Flower.crossover_helper_middleOfColors (parent1, parent2, cutPoint)
            print("----- Crossover child 1 -----")
            child1.printDetails()
            print("----- Crossover child 2 -----")
            child2.printDetails()
            return child1, child2
        else :
            child1, child2 = Flower.crossover_helper_endOfColors (parent1, parent2, cutPoint)
            print("----- Child 1 -----")
            child1.printDetails()
            print("----- Child 2 -----")
            child2.printDetails()
            return child1, child2
```

Mutation

Mutation randomly alters genes to introduce new variability into the population.

- The `Flower.mutate` method is applied to every child produced by the crossover step.
- A random number from 0 to 100 is generated. If this number is less than 5, mutation occurs. This implements the required 5% mutation probability.
- If mutation occurs, a single gene to mutate (an integer from 1 to 11) is randomly selected.
- If the gene is `sizeOfCenter` (1) or `numberOfPetals` (11), it is replaced with a new, valid random value.
- If the gene is a color component (2-10), the `mutate_color_component` helper is called. This function replaces just the single R, G, or B value with a new random value from 0 to 255.
- Detailed print statements are included to show the flower's characteristics before and after the mutation.

```
@staticmethod
def mutate(flower):
    print("*** Is Mutation Occurring? ***")
    mutation_chance = random.randint(0, 100)
    if mutation_chance < 5 :

        gene_to_mutate = random.randint(1,11)
        print(f"*** Mutation occurred on flower at gene {gene_to_mutate} ***")
        if 2 <= gene_to_mutate <=10 :
            flower = Flower.mutate_color_component(flower, gene_to_mutate)
        else :
            # Mutate sizeOfCenter or numberOfPetals
            if gene_to_mutate == 1:
                new_size = random.uniform(1,3)
                flower.setSizeOfCenter(new_size)
                print(f"--- Mutation in Size of Center to {new_size} ---")
                flower.printDetails()
            elif gene_to_mutate == 11:
                new_number = random.randint(1, 7)
                flower.setNumberOfPetals(new_number)
                print(f"--- Mutation in Number of Petals to {new_number} ---")
                flower.printDetails()
        else :
            print("*** No Mutation Occurred ***")
            flower.printDetails()

    return flower
```

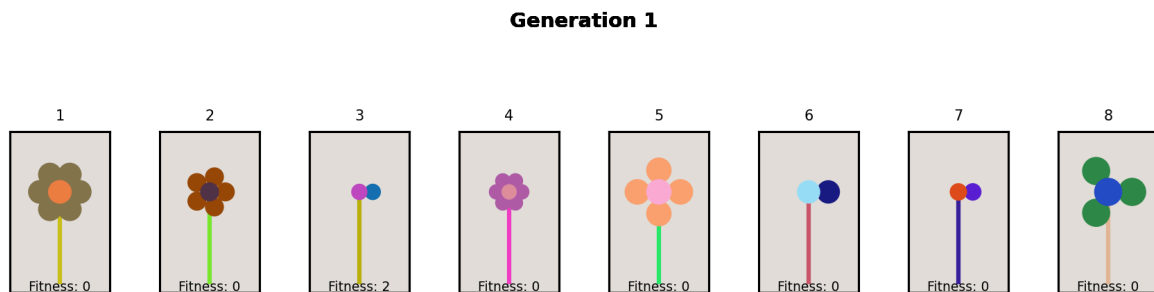
Evaluation and Results

This section provides snapshots of the simulation and the console output, visualizing the algorithm.

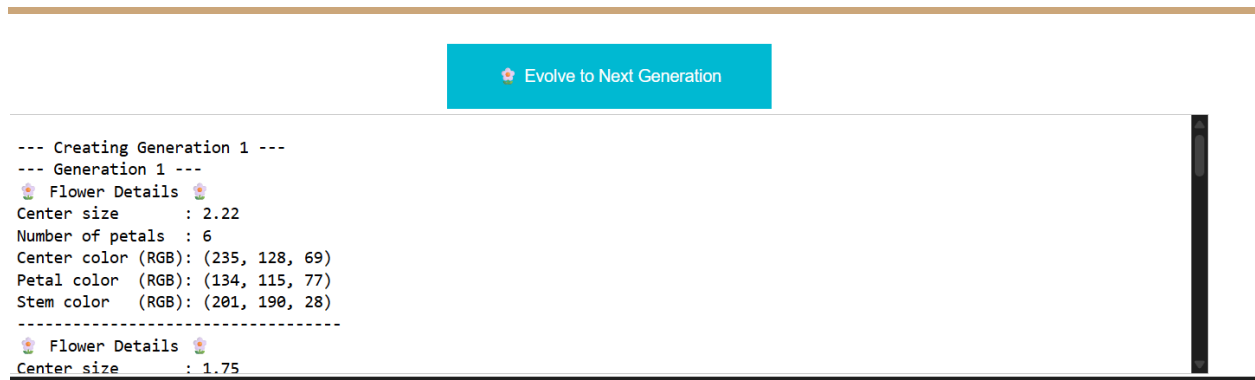
Initial Population

The simulation begins by clicking the "Create First Generation" button, which calls the `initPop` function. This generates the initial, random population of 8 flowers.

GUI - Generation 1: The image below displays the 8 randomly generated flowers for Generation 1. As required, each flower has a unique set of characteristics (DNA) and starts with a fitness of 0.



Console Output - Generation 1: Simultaneously, the console prints the detailed characteristics (genes) for each flower created in this initial population, as seen below.



After Fitness Evaluation

The user then interacts with the population. In the "Generation 1" snapshot, you can see that the user has hovered over flower #3, increasing its **Fitness to 2**. This flower, having the highest fitness, will be favored during the selection process for creating Generation 2.

Crossover

When the "Evolve to Next Generation" button is clicked, the `evolve` function is triggered. This begins the process of selection, crossover, and mutation, all of which are logged to the console.

Crossover - Parents: The console output below shows a crossover event occurring at **gene 2**. It prints the full details of the two parent flowers selected for reproduction.

```

*** Crossover at gene 2 ***
---- Parent 1 ----
🌸 Flower Details 🌸
Center size      : 2.22
Number of petals : 6
Center color (RGB): (235, 128, 69)
Petal color  (RGB): (134, 115, 77)
Stem color   (RGB): (201, 190, 28)
-----
---- Parent 2 ----
🌸 Flower Details 🌸
Center size      : 1.75
Number of petals : 6
Center color (RGB): (235, 128, 69)
Petal color  (RGB): (134, 115, 77)
Stem color   (RGB): (201, 190, 28)
-----

```

Crossover - Child: Following the parent details, the console shows the resulting children. The screenshot below shows the details for "Crossover child 2".

- **Parent 1's** `sizeOfCenter` was 2.22.
- **Parent 2's** `numberOfPetals` was 4.

The new "child 2" has inherited the `numberOfPetals` from Parent 2 (value: 6) and the `sizeOfCenter` from a different parent (value: 2.4), demonstrating a successful crossover.

```
🌺 FLOWER DETAILS 🌺
enter size      : 2.22
umber of petals : 4
enter color (RGB): (235, 171, 210)
etal color  (RGB): (254, 161, 110)
tem color   (RGB): (40, 234, 108)
-----
----- Crossover child 2 -----
🌺 Flower Details 🌺
enter size      : 2.4
umber of petals : 6
enter color (RGB): (252, 128, 60)
```

Generation 2

After selection, crossover, and mutation are applied to create 8 new children, this new population replaces the old one. The GUI updates to display "Generation 2". As shown below, all fitness scores are reset to 0, ready for the next round of user-based fitness evaluation.

Visually, we can already see the influence of the "fitter" flowers from Generation 1. For example, flower #2 in Generation 2 (orange petals, pink center) looks very similar to flower #5 from Generation 1, and flower #8 (brown petals, orange center) resembles flower #1 from Generation 1, indicating their traits were likely passed on.

Generation 2

